

The architecture I would build

The central mistake is to treat this as a generic speech-denoising problem. It is primarily an **acoustic echo cancellation and synchronization problem**.

When you are not using headphones, the remote speaker exists in two places:

1. The clean digital system-audio stream.
2. A delayed, filtered, reverberant copy of that stream picked up by the microphone.

If playback is effectively:

```
[
\text{recording} = \text{system audio} + \text{raw mic}
]
```

then the remote participant will necessarily sound echoed. The desired recording is:

```
[
\text{recording} = \text{system audio} + \text{AEC}(\text{raw mic}, \text{speaker reference})
]
```

where the echo canceller receives the **exact signal actually sent to the speakers**, correctly synchronized with the microphone.

My recommended production stack is:

Core Audio taps or a virtual speaker → exact render reference → clock/delay alignment → WebRTC AEC3 → conservative full-band noise enhancement → a stronger offline post-call pass → separate near/far transcription.

The model is not the most important part. A perfect model with the wrong reference or a drifting timeline will perform badly. A mature model with the correct topology can sound excellent.

1. First determine which problem you actually have

These problems sound similar but require different fixes.

Symptom	Likely cause	Correct treatment
Remote participant sounds doubled at a fairly constant delay	Clean system stream mixed with remote leakage in raw mic	AEC the mic before mixing
Echo starts acceptable and worsens over a long call	Mic and render clocks are drifting	Continuous resampling and delay tracking
Echo appears after changing volume or output device	Echo path changed	Reset/reconverge AEC; crossfade
Your own voice sounds hollow or like an empty room even when nobody else speaks	Room reverberation	Dereverberation after AEC
Voice sounds metallic, thin, underwater, or phasey	Over-suppression or stacked processors	Disable redundant NS/AEC/AGC and reduce attenuation
Only the beginning of phrases echoes	AEC adaptation or VAD/gating onset	Maintain continuous AEC state; avoid hard gates

Echo happens around notifications or music but not the call audio	AEC reference does not include everything the speakers played	Use a full physical-render reference, not only the call-process stream
---	---	--

A fast diagnostic is:

1. Record a segment where only the remote person speaks.
2. Listen to the system track by itself.
3. Listen to the mic track by itself.
4. Listen to your current mix.
5. Cross-correlate the system track against the mic track.

You will probably find a strong delayed, room-colored copy of the system audio in the mic track. That is precisely the signal the AEC should remove.

2. Use four logical stems, not two

I would retain at least these streams:

Stem	Purpose
far_program_raw	The clean call application output used in the final recording and remote transcription
render_reference_raw	Everything actually sent to the physical speakers, used only as the AEC reference
mic_raw	Untouched microphone capture, retained permanently
near_clean	Mic after AEC, residual echo suppression, and conservative enhancement

The subtle distinction between the first two is important.

Suppose you capture only Zoom or Chrome as `far_program_raw`, but macOS plays a notification. The microphone hears the notification, yet your AEC reference does not contain it. The echo canceller cannot remove something absent from its reference.

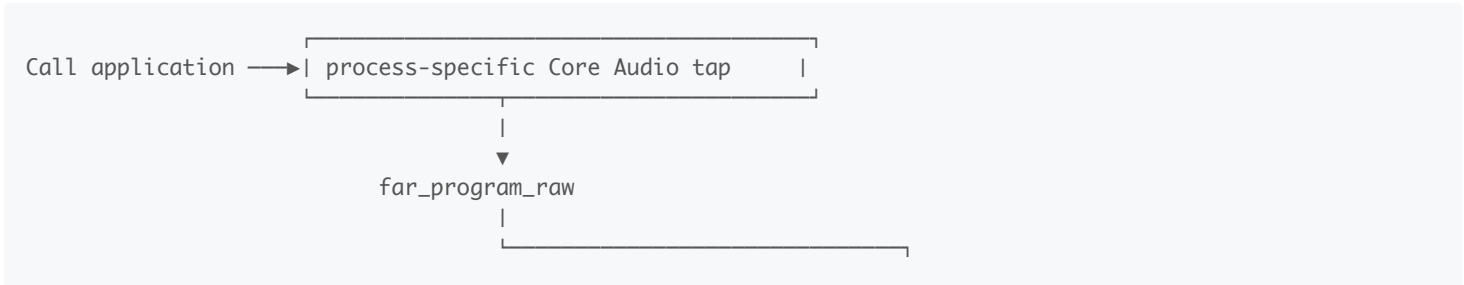
Therefore:

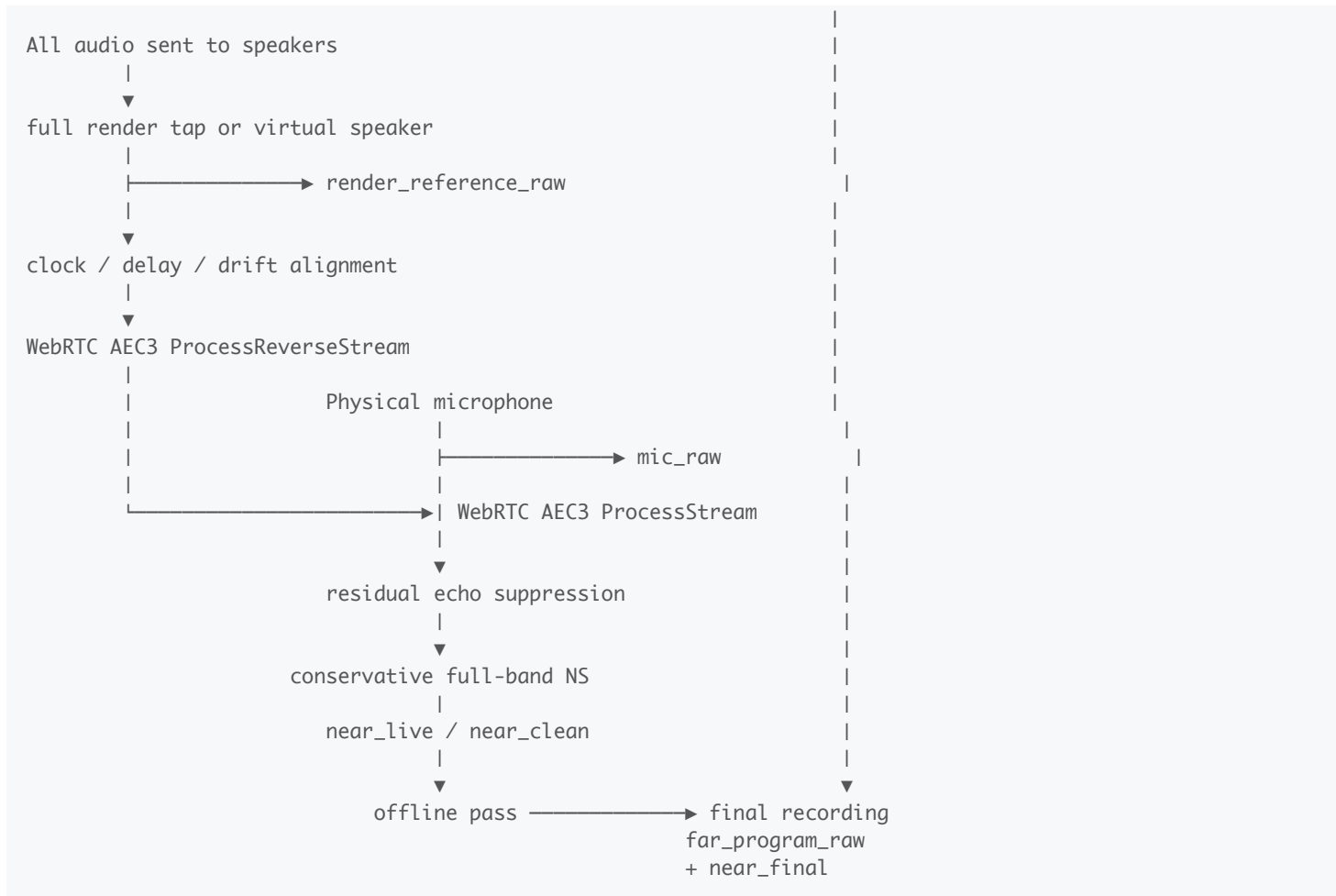
- **Final playback:** `far_program_raw + near_clean`
- **AEC reference:** ideally the complete physical output render
- **Never:** `far_program_raw + mic_raw`

When the call application is the only source playing audio, `far_program_raw` and `render_reference_raw` can be the same captured signal.

Apple’s current Core Audio tap APIs can capture outgoing audio from an individual process or group of processes, provide mono or stereo mixdowns, and expose the tap through a HAL aggregate device. Apple’s sample requires macOS 14.2 or later and the system-audio capture usage description and permission. ([Apple Developer](#))

A high-level graph:





3. Synchronization is probably the hardest engineering problem

Model quality will collapse when the reference and mic are even modestly misaligned.

Represent the microphone signal as:

$$[m[n] = s[n] + h_n * g(x[n - \Delta_n]) + v[n]]$$

where:

- $s[n]$ is your local voice.
- $x[n]$ is the digital render reference.
- Δ_n is time-varying system and acoustic delay.
- h_n is the changing room/speaker/microphone impulse response.
- g captures speaker distortion, limiting, EQ, and other nonlinearities.
- $v[n]$ is background noise.

You need to estimate both:

1. **The initial offset:** output buffering, device latency, propagation, tap placement.
2. **The ongoing rate difference:** independent or imperfectly synchronized clocks.

Even 50 ppm of drift amounts to about 180 milliseconds over an hour. That is enough to turn successful cancellation into obvious echo.

Capture timestamps, not callback order

Every audio block should carry:

```
stream_id
host_time
device_sample_time
nominal_sample_rate
frame_count
channel_count
device_uid
route_generation
output_volume_generation
dropped_frame_count
```

Use Core Audio's host and sample timestamps. Do not align streams based on when their callbacks happen to arrive on your thread.

I would use one canonical 48 kHz timeline and map every input onto it. For a stream (i):

```
[
t_{\text{master}} = a_i \cdot n_i + b_i
]
```

where (b_i) is offset and (a_i) captures the actual sample-rate ratio. Update (a_i) slowly and continuously rather than occasionally inserting or deleting whole samples.

Combine timestamp alignment with audio-derived alignment

Host timestamps give a strong initial estimate, but they do not fully identify the acoustic delay. Use:

- Host/sample timestamps for coarse positioning.
- GCC-PHAT or cross-spectrum coherence for acoustic delay.
- Far-end-only regions for reliable estimation.
- Robust smoothing or a Kalman-style tracker for delay over time.
- A fractional resampler to correct drift.
- Frozen or heavily damped delay updates during double-talk.

For offline processing, fit a piecewise affine mapping between the render and microphone timelines. Start a new segment after:

- Output-device changes.
- Input-device changes.
- Sample-rate changes.
- Bluetooth reconnections.
- Sleep/wake.
- Long callback gaps.
- Large output-volume or balance changes.

Do not silently close gaps in either track. Insert timestamped silence so that the common timeline remains truthful.

4. WebRTC AEC3 should be your production baseline

I would start with **WebRTC Audio Processing Module's AEC3**, not a monolithic neural echo canceller.

WebRTC's Audio Processing Module is explicitly designed around a near-end capture stream passed to `ProcessStream()` and a far-end render stream passed to `ProcessReverseStream()`. It expects 10 ms linear PCM frames and recommends placement close to the hardware abstraction layer. The documented usage processes the render frame, sets the stream delay, and then processes the capture frame. Its current configuration supports 48 kHz internal processing, multichannel render, device/volume change notifications, and exporting the linear AEC output for subsequent processing. ([WebRTC Git Repositories](#))

A reasonable initial configuration is conceptually:

```
webrtc::AudioProcessing::Config config;

config.pipeline.maximum_internal_processing_rate = 48000;
config.pipeline.multi_channel_render = true;    // useful for stereo speakers

config.echo_canceller.enabled = true;
config.echo_canceller.mobile_mode = false;
config.echo_canceller.export_linear_aec_output = true;

// Disable these while establishing the AEC baseline.
config.noise_suppression.enabled = false;
config.gain_controller1.enabled = false;
config.gain_controller2.enabled = false;

// Add only if microphone rumble actually requires it.
config.high_pass_filter.enabled = false;

apm->ApplyConfig(config);
```

Then for each aligned 10 ms block:

```
apm->ProcessReverseStream(render_frame);
apm->set_stream_delay_ms(current_delay_ms);
apm->ProcessStream(mic_frame);
```

Pin a particular WebRTC commit or release in your build because its APIs and defaults evolve.

Stereo versus mono reference

The MacBook speakers are stereo and have different acoustic paths to the microphone. AEC3 supports multichannel render, so I would initially preserve two render channels.

A mono reference is acceptable when:

- The call output is mono duplicated across left and right.
- You have tested it against stereo and found no regression.
- Integration simplicity matters more than difficult edge cases.

Avoid an uninformed $0.5 * (L + R)$ downmix when audio could contain anti-phase or spatial components.

Process order

Use this ordering:

```
mic
→ optional DC/very-low-frequency removal
→ linear AEC
→ residual echo suppression
→ noise suppression
→ optional dereverberation
→ leveler/compressor
→ limiter
```

Do **not** put neural denoising before AEC. Denoising changes the microphone signal in a nonlinear, time-varying way and makes adaptive echo-path estimation harder.

Initially enable only AEC3. Once it is stable and the remaining defects are understood, add exactly one later processor at a time.

5. The option ladder

Rung 0 — Correct the recording topology

Components: raw stems, correct final mix, timestamps, no new model.

This alone may eliminate most of the perceived echo. It is mandatory regardless of later choices.

Quality ceiling: high when speaker leakage is low or there is little double-talk.

Engineering effort: low.

Rung 1 — Apple Voice Processing I/O

Use AVAudioEngine's voice-processing mode or the underlying Voice Processing I/O audio unit when your application controls both microphone capture and speaker rendering.

Apple describes its voice-processing mode as intended for echo cancellation and VoIP. Both input and output nodes must be in voice-processing mode, it only operates while rendering to an audio device rather than manual rendering, and it cannot be switched dynamically while the engine is running. Apple also warns that its real-time audio blocks must not allocate memory, dispatch work, or block on mutexes. ([Apple Developer](#))

Advantages

- Very little DSP integration.
- Tuned for Apple devices.
- Potentially benefits from Apple's knowledge of built-in hardware.
- Excellent benchmark against your own stack.

Limitations

- Opaque behavior and limited tuning.
- You cannot straightforwardly hand it an arbitrary captured stream from another application as its far-end reference.
- It works best when the call audio actually flows through your AVAudioEngine.
- It may apply tonal processing you do not want in a high-fidelity archive.

This is the fastest baseline, but not my preferred architecture for a recorder that merely taps another application.

Rung 2 — Core Audio taps + WebRTC AEC3

Components

- Core Audio process tap for the call.
- Optional second full-render tap.
- AUHAL or AVAudioEngine for mic capture.

- Explicit timestamp, delay, and drift layer.
- WebRTC AEC3.
- Raw and cleaned stem recording.

This is my recommended first production version.

Advantages

- Mature AEC.
- Full-band 48 kHz processing.
- Open and locally deployable.
- Good control and observability.
- No mandatory virtual device.

Limitations

- Capturing a pre-output process stream may not perfectly represent every transformation between the application and the physical speaker.
- Route changes and separate clocks require substantial engineering.
- It does not give you the same end-to-end routing control as Krisp.

Rung 3 — Add full-band enhancement and a separate offline pass

After AEC3, use **DeepFilterNet3** conservatively for background noise and mild residual artifacts.

DeepFilterNet is designed for full-band 48 kHz speech enhancement, has a real-time implementation, supports macOS, and is dual-licensed under MIT or Apache 2.0. Its repository includes DeepFilterNet3 and identifies it as perceptually motivated real-time enhancement. ([GitHub](#))

I would use:

- AEC3 in real time.
- DeepFilterNet3 after AEC3.
- Its aggressive postfilter disabled initially.
- A user-facing strength control internally capped below the model's most aggressive setting.
- A separate post-call rerender with improved alignment.

This is likely the best near-term balance between fidelity and implementation risk.

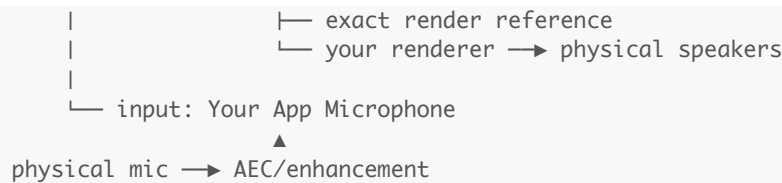
Rung 4 — Krisp-like virtual speaker and microphone

For the highest topology-controlled quality, expose:

- Your App Speaker
- Your App Microphone

The call application sends remote audio to your virtual speaker. Your app receives those exact samples, forwards them to the physical speaker, and uses the same samples as the AEC reference. Your app captures the physical mic, cleans it, and optionally publishes it through the virtual microphone.

```
Conference app
  |
  └─ output: Your App Speaker
  |
  |
```



Apple provides an Audio Server Driver Plug-in sample specifically for building a virtual audio device. The sample supports 44.1/48 kHz and 32-bit floating-point PCM. Apple separately recommends Audio Server Driver Plug-ins, rather than AudioDriverKit, for virtual devices. ([Apple Developer](#))

Why this improves quality

- You know the exact digital render signal.
- You own sample-rate conversion and buffering.
- You can make the output reference and microphone timeline share one clock domain.
- You can notify the AEC immediately about device and volume changes.
- You avoid relying on an approximate process tap.
- You can provide the cleaned signal to any call application.

Costs

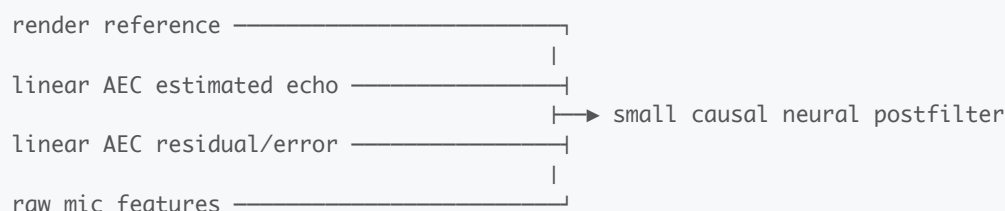
- Driver installation and update lifecycle.
- Signing, notarization, permissions, installer, and uninstaller.
- Shared-memory or IPC design between driver and app.
- More difficult crash recovery and device-switch handling.
- Higher support burden.

For a serious product intended to match Krisp, I would ultimately build this rung.

Rung 5 — Custom hybrid residual echo model

Only after AEC3 and synchronization are demonstrably correct would I train a custom model.

The most practical architecture is:



The neural model should not be responsible for discovering the entire acoustic path from scratch. Let an adaptive filter remove the easy linear component; use the neural stage for:

- Loudspeaker nonlinearities.
- Residual reverberant echo.
- Echo-path changes.
- Difficult double-talk.
- Joint echo and noise suppression.
- Near-end speech preservation.

This hybrid structure remains a strong industrial choice. Even a 2026 end-to-end AEC paper notes that hybrid systems continue to predominate in industrial deployment because of their robustness and reliability. ([arXiv](#))

Rung 6 — License a commercial SDK

Krisp currently offers a Mac-capable RTC SDK for human-to-human calls with inbound and outbound noise cancellation and related processing. Its public product page does not clearly state that a separately exposed acoustic-echo-cancellation component is included in every SDK configuration, so that needs to be confirmed during technical evaluation. ([Krisp](#))

This can be a rational option when:

- Time to quality matters more than owning the core DSP.
- Their licensing permits fully local operation and your privacy model.
- Their AEC interface accepts a far-end reference rather than only a mic stream.
- You can retain your own raw stems.
- Telemetry and model-update behavior are contractually clear.

Even with an SDK, you still need correct capture, synchronization, and mixing.

6. What I think Krisp is doing

Krisp's exact model architecture is proprietary, so no public source establishes every internal stage. But its public behavior strongly reveals the signal topology.

Krisp says both **Krisp Microphone** and **Krisp Speaker** must be selected for its acoustic echo cancellation to operate. It also distinguishes acoustic echo cancellation—speaker output returning through the mic—from room echo cancellation or dereverberation. ([Krisp Help](#))

That implies a system approximately like:

```
virtual speaker
  → exact far-end/render access
  → output path
  → physical speaker

physical mic
  → adaptive/neural AEC using render
  → noise suppression
  → dereverberation
  → voice-preservation and level processing
  → virtual microphone
```

Krisp publicly describes a combination of custom neural architecture with DSP preprocessing and postprocessing. It says its models operate on small streaming chunks, are trained on several thousand hours of clean speech and noise, use augmentations covering microphones, acoustic conditions, signal-to-noise ratios, and bandwidths, and achieve under 20 ms algorithmic latency. ([Krisp](#))

It also explicitly advises avoiding extra onboard or software processing because a second processor on the same signal can degrade quality. ([Krisp Help](#))

So Krisp's quality advantage is probably a combination of:

1. **Correct bidirectional virtual routing.**
2. **A very large and diverse proprietary training corpus.**
3. **Device- and room-specific augmentation.**
4. **A classical or hybrid echo-path front end.**
5. **A neural residual/noise/dereverb stage.**
6. **Extensive double-talk and near-speech-preservation tuning.**

7. **Conservative output mastering.**
8. **Years of QA against real conferencing applications and hardware.**

The “warm” sound is likely as much about **what Krisp elects not to remove** as what it removes. Aggressive spectral suppression creates the metallic quality you are hearing.

7. Model recommendations

Acoustic echo core: WebRTC AEC3

Recommendation: Production default.

Why:

- Actual far-end-reference AEC, not generic denoising.
- Mature handling of double-talk and echo-path changes.
- 48 kHz and multichannel-render options.
- BSD-style license.
- Linear output can be exposed for a custom residual stage. ([WebRTC Git Repositories](#))

Do not tune obscure AEC3 internals until delay, clocks, and route changes are correct.

Noise suppression: DeepFilterNet3

Recommendation: Best first open-source full-band enhancement candidate.

Why:

- 48 kHz rather than telephone-band processing.
- Real-time implementation.
- macOS support.
- Commercially usable MIT/Apache licensing.
- Generally more appropriate for natural playback than a 16 kHz joint AEC model. ([GitHub](#))

Use it after AEC, not instead of AEC.

Experimental joint AEC: LocalVQE

LocalVQE is one of the more directly usable recent open implementations. Its current models operate at 16 kHz with a 256-sample/16 ms streaming hop. Its v1.4-AEC model combines a classical adaptive filter with a small neural mask; the project also offers joint AEC, noise suppression, and dereverberation variants. It requires a separate far-end reference and is Apache 2.0 licensed. The published benchmarks are project-reported, and its OBS integration has been tested on Linux while macOS is described only as expected to work. ([GitHub](#))

I would test it for:

- An ASR-clean track.
- A low-complexity A/B benchmark.
- Residual-mask architecture ideas.
- Cases where you intentionally want joint AEC/NS/dereverb.

I would **not** make it the default final-playback processor because 16 kHz sampling limits the retained spectrum to 8 kHz. That can directly contribute to the narrow, telephone-like quality you are trying to avoid.

Research watchlist, not immediate dependencies

EchoFree

EchoFree combines linear filtering with a neural postfilter and reports 278,000 parameters and 30 MMACs, with results comparable to lightweight DeepVQE variants on the ICASSP AEC test set. Its public repository says code and models are intended to be released following paper acceptance, so it is currently more useful as an architectural reference than a shipping dependency. ([arXiv](#))

LMPAN

LMPAN, published in July 2026, uses separate alignment paths for the reference, linear-AEC output, and microphone, followed by adaptive fusion and a downstream-oriented postfilter. That design directly addresses the practical problem of temporal and energy mismatch between the three signals. The paper reports a 480,000-parameter on-device architecture, but it is too recent to treat as a battle-tested implementation. ([arXiv](#))

MSA-EchoLite

MSA-EchoLite appeared in August 2026. It combines linear AEC with a compact Bark-domain neural system and reports 0.2 million parameters and approximately 100 million FLOPs per second. It is an interesting design for an eventual custom residual stage but is far too new to replace AEC3 in a production recorder. ([arXiv](#))

E2E-AEC

E2E-AEC uses streaming attention-based alignment rather than a traditional time-delay estimator and linear adaptive filter. It is notable research, particularly for difficult changing delays, but it does not currently provide the same deployment confidence as a hybrid AEC architecture. ([arXiv](#))

8. A stronger offline post-call pass

Because the high-quality audio is only needed for playback after the call, you should exploit non-real-time processing. The live path should optimize latency and ASR stability; the offline path should optimize naturalness and echo removal.

Offline pipeline

1. Reconstruct a truthful common timeline

- Read timestamped raw mic and render chunks.
- Restore dropped-frame gaps as silence.
- Split at route changes and discontinuities.
- Do not begin with the already mixed audio.

2. Estimate delay and drift globally

For each continuous segment:

- Find far-end-only intervals.
- Compute GCC-PHAT or coherence delay estimates over overlapping windows.

- Reject low-confidence and double-talk estimates.
- Fit a smooth offset curve or piecewise affine timeline.
- Fractionally resample the reference onto the mic clock.

This offline realignment may improve quality more than switching models.

3. Run a long-tail adaptive filter

The built-in-speaker-to-mic impulse response can include substantial room reverberation. An offline partitioned-block frequency-domain adaptive filter can use a longer tail—perhaps 0.5 to 2 seconds—than you would comfortably use in a minimal-latency path.

Important safeguards:

- Freeze or slow adaptation during double-talk.
- Reset at route transitions.
- Crossfade around filter resets.
- Prime the filter with render-only context before exposing output.
- Detect large output gain changes.

You can also rerun AEC3 faster than real time after correcting alignment. A custom long-tail filter is useful primarily when AEC3's real-time assumptions are limiting you.

4. Apply a neural residual postfilter

Inputs should include:

- Aligned far-end spectrum.
- Raw microphone spectrum.
- Linear echo estimate.
- Linear-AEC error signal.
- Double-talk or near-VAD features.
- Current echo-path confidence.
- Recent delay and output-volume state.

Output either:

- A complex ratio mask.
- ERB/Bark-band gains.
- A bounded complex spectral correction.

Bias the model toward **preserving near speech**. A small amount of residual room sound is less objectionable than metallic consonants.

5. Noise suppression and dereverberation

Only after echo cancellation:

- Full-band denoising.
- Mild dereverberation.
- Keyboard/transient reduction.
- Slow level matching.

I would maintain two post-call products:

- **Faithful:** conservative, suitable for archival and review.
- **Studio:** stronger enhancement or restoration, explicitly labeled.

9. Avoid generative restoration in the canonical recording

ClearerVoice-Studio provides open-source models for 16 and 48 kHz speech enhancement, separation, and bandwidth extension under Apache 2.0. It is a reasonable offline comparison candidate. ([GitHub](#))

Resemble Enhance combines a denoiser with an enhancement model that restores distortions and extends bandwidth; its enhancer uses a learned autoencoder/vocoder and conditional flow model trained on 44.1 kHz speech. ([GitHub](#))

These systems can make severely damaged audio sound subjectively impressive. But restoration and bandwidth extension may synthesize speech detail that was not present in the recording. Therefore:

- Do not use generative restoration for the raw archive.
- Do not use it as the sole input to transcription without measuring WER.
- Do not use it where exact wording or evidence fidelity matters.
- Offer it as an optional playback render.
- Keep one-click access to the faithful version.

For a call-review product, a slightly noisy but faithful consonant is generally better than a beautifully reconstructed wrong consonant.

10. How to get “warm” rather than metallic

“Warm” speech usually comes from **retaining useful low-mid information and avoiding temporal artifacts**, not from applying a large bass boost.

Preserve full bandwidth

- Capture and process at 48 kHz.
- Store float32 or 24-bit PCM until final export.
- Do not route final playback through a 16 kHz model.
- Do not process the clean far-end track unless necessary.

The remote track has already undergone the conference application's codec, noise processing, and potentially AGC. Running it through another mic-oriented enhancer often makes it worse.

Use conservative spectral processing

For the local stem, a reasonable starting mastering chain is:

```
optional 50–70 Hz high-pass
→ gentle dynamic EQ
→ mild de-essing only when needed
→ slow compressor
→ true-peak limiter
```

Suggested starting values, not universal presets:

- High-pass: around 50–70 Hz, 12 dB/octave.
- Avoid high-passing at 120–150 Hz unless the mic is extremely boomy.
- Compression: approximately 1.5:1 to 2:1.
- Gain reduction: usually 2–4 dB on normal speech.

- Attack: approximately 20–40 ms.
- Release: approximately 100–250 ms.
- Final true peak: around –1 dBTP.
- Playback mix: approximately –16 to –18 LUFS integrated, depending on product context.

Do not use:

- Hard noise gates.
- Aggressive spectral-floor removal.
- Automatic EQ that changes radically phrase by phrase.
- Excessive dereverberation.
- Blanket presence boosts around 2–5 kHz.
- Strong AGC before AEC.

A little stable background ambience is less perceptually disturbing than noise that pumps in and out between every syllable.

Protect near-end-only speech

During a segment where only you are talking and the speaker reference is silent, `near_clean` should remain extremely close to `mic_raw`.

That is one of the most important regression tests. An AEC or residual suppressor that heavily changes near-end-only speech is overprocessing.

11. Transcribe each side separately

Since you already possess distinct system and mic signals, do not transcribe the final mixture unless it is merely a fallback.

Use:

```
near transcript = ASR(near_asr_clean)
far transcript  = ASR(far_program_raw)
conversation   = merge by common timestamps
```

Benefits:

- Your local identity is known automatically.
- No near-versus-far diarization is needed.
- Overlapping speech is much easier.
- Echo in the mic does not create duplicated words.
- Different enhancement settings can be used for playback and ASR.
- Remote diarization is needed only within the far-end track when multiple remote people share it.

Apple SpeechAnalyzer

Apple’s SpeechAnalyzer and SpeechTranscriber are on-device APIs introduced across Apple platforms in OS 26. Apple describes the model as suited to long-form, distant audio such as meetings and conversations. It supports sample-precise timecodes, fast volatile results followed by finalized results, and system-managed model assets. ([Apple Developer](#))

It is appealing for:

- Native Swift integration.
- Live UI transcription.
- Precise timeline integration.
- Low operational complexity.
- Local processing.

The tradeoffs are OS-version requirements, limited model control, and system-provided model updates that may affect strict reproducibility.

whisper.cpp

`whisper.cpp` is optimized for Apple Silicon through ARM NEON, Accelerate, Metal, and Core ML. It supports offline inference, runtime allocation-free execution, quantization, and GPU/ANE paths. ([GitHub](#))

OpenAI's current Whisper repository describes `turbo` as an optimized `large-v3` variant with approximately 809 million parameters and roughly 6 GB of required VRAM, offering much faster inference with minimal accuracy degradation relative to the larger model. ([GitHub](#))

For your M1 Max:

- Run AEC and alignment on CPU.
- Run live ASR on Metal or ANE.
- Keep real-time audio threads isolated from ASR scheduling.
- Use a smaller or Apple-native model for volatile live output.
- Rerun the final stems with `turbo` or `large-v3` after the call.
- Pin model files and inference settings for reproducibility.

Your 64 GB unified memory is ample for this stack. The meaningful constraint will be p99 scheduling latency and thermal/contention behavior, not model memory.

12. Real-time implementation details

Audio callback

The callback should do only:

- Copy or reference PCM into a preallocated ring.
- Record the hardware timestamps.
- Advance atomic indexes.
- Return.

No:

- File writes.
- Swift object allocation.
- Logging.
- Model inference.
- Mutex waits.
- Dispatching synchronous work.

Apple explicitly notes that real-time audio blocks should avoid allocations, dispatch calls, and blocking on mutexes. ([Apple Developer](#))

Thread layout

```
Core Audio render callback
  → lock-free render ring

Core Audio mic callback
  → lock-free mic ring

high-priority DSP thread
  → timeline alignment
  → AEC3
  → live enhancement
  → ASR input ring
  → clean recorder ring

writer thread
  → raw stems
  → clean stem
  → metadata journal

ASR worker
  → Metal / Core ML / ANE

offline worker after call
  → global alignment
  → final AEC
  → enhancement/mastering
```

Use separate rings for archival recording and live processing so a slow disk or ASR worker cannot interrupt AEC.

Storage

CAF is convenient for long Core Audio recordings, while FLAC or ALAC can reduce post-call storage. Regardless of container:

- Retain the original sample count and timestamps.
- Journal metadata incrementally.
- Periodically make file headers recoverable.
- Never store only the processed mix.
- Maintain a route-change event log.

13. Training a custom Krisp-like residual model

Once the baseline is working, construct synthetic and real training examples:

```
[
m = s_near + h * g(x_far) + n
]
```

Data sources

Microsoft's open AEC Challenge includes real and synthetic recordings from more than 10,000 devices and human speakers, covers single-talk and double-talk, includes full-band AECMOS tooling, and used both mean opinion score and word accuracy to select winners. ([GitHub](#))

Add your own Mac-focused dataset:

- Multiple MacBook generations.
- Built-in and external displays.
- Built-in, USB, AirPlay, and Bluetooth output.
- Different output volumes.
- Different desk materials and distances to walls.
- Furnished and unfurnished rooms.
- User movement.
- Lid angles.
- Speaker balance and spatial-audio settings.
- Zoom, Meet, Teams, FaceTime, browser audio, music, and notifications.

Record a close-talk reference mic or lavalier as the near-speech target while the laptop mic captures the mixed signal.

Augmentations

Randomize:

- Delay from 0 to several hundred milliseconds.
- Sample-rate drift.
- Buffer jitter and occasional gaps.
- Linear and nonlinear loudspeaker response.
- Soft clipping and limiting.
- Output EQ.
- Room impulse response and moving echo paths.
- Signal-to-echo ratio.
- Signal-to-noise ratio.
- Double-talk.
- Near and far speech in multiple languages.
- Music, ringtones, and non-speech render content.
- Sudden output-volume changes.
- Device switches.

Losses

Use a mixture of:

- Multi-resolution STFT loss.
- Complex spectral loss.
- Scale-invariant SDR.
- Echo-aware far-end-only loss.
- Near-speech-preservation loss.
- Double-talk loss.
- Perceptual or self-supervised speech-feature loss.
- ASR loss or measured WER in validation.
- Penalty for time-varying musical artifacts.

Do not optimize only ERLE. A model can obtain enormous echo attenuation by deleting everything, including the local voice.

14. Evaluation framework

Create a fixed regression corpus with:

- Near-end-only speech.
- Far-end-only speech.
- Double-talk.
- Double-talk with user movement.
- Speaker volume jumps.
- Device switching.
- Music and notifications.
- Keyboard and desk impacts.
- Quiet and noisy rooms.
- External monitors and speakers.
- Bluetooth delay changes.
- Long recordings for drift.

Measure:

Metric	What it tells you
AECMOS	Perceived echo and other degradation
ERLE	Echo attenuation during far-end-only speech
Near-only SI-SDR or spectral distance	How much you damage the local speaker
WER / word accuracy	Downstream transcription quality
Residual reference coherence	Remaining far-end content in the cleaned mic
NISQA coloration/discontinuity	Metallic or temporal artifacts
Dropouts and p99 processing time	Product reliability
Human A/B or MUSHRA-style test	Actual listener preference

NISQA provides overall quality plus noisiness, coloration, discontinuity, and loudness estimates, which makes it useful for detecting the “tin” and instability you described. Its code is MIT, but the supplied model weights use a noncommercial Creative Commons license, so treat it as an internal evaluation tool unless licensing is resolved. ([GitHub](#))

The final selection should be based on human preference and WER together, not one objective metric. Microsoft’s AEC Challenge similarly used both MOS and word accuracy. ([GitHub](#))

15. Exactly what I would implement first

Initial production candidate

Capture

- 48 kHz float32.
- `far_program_raw` : process-specific Core Audio tap.
- `render_reference_raw` : full-output tap when feasible.
- `mic_raw` : AUHAL physical microphone capture.

- Timestamp every buffer with host and sample time.

Live DSP

- Canonical 48 kHz master timeline.
- Continuous fractional resampling.
- Host-time initial alignment.
- GCC-PHAT-assisted acoustic delay.
- WebRTC AEC3, stereo render reference when useful.
- WebRTC NS and AGC disabled.
- DeepFilterNet3 after AEC at conservative strength.
- Mild live limiter only if needed for ASR.

Recording

- Raw mic.
- Raw far program.
- Raw render reference.
- Live-clean mic.
- Route and delay metadata.

Transcription

- Near and far stems separately.
- SpeechAnalyzer for native live text when the deployment target permits.
- whisper.cpp `turbo` or `large-v3` for final pass.
- Merge results using the common sample timeline.

Offline render

- Re-estimate delay and drift globally.
- Rerun AEC with corrected alignment.
- Long-tail adaptive filter where needed.
- Conservative residual and noise suppression.
- Mild dereverberation.
- Gentle mastering.
- Export `Faithful` and optionally `Studio` versions.

Later product-quality upgrade

Replace the render tap with a virtual speaker device so that you own the exact output buffer and clock. Add a virtual microphone only when you also want to improve what other participants hear.

That sequence gets you the benefits in the correct order:

1. Fix duplicate mixing.
2. Fix timestamps and drift.
3. Establish AEC3.
4. Add conservative full-band enhancement.
5. Exploit offline processing.
6. Control routing with a virtual device.
7. Train a custom neural residual model.

The highest-leverage immediate change is therefore not a larger model. It is recording **the exact render reference, the**

untouched mic, and the call program as separate, correctly timestamped streams, then building the final audio only from the clean program track and an echo-cancelled local track.